

Módulo 1: Fundamentos y Entorno

T03: VulnApp Setup

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

 *Conoce la app que vas a hackear — y reparar*

Objetivos

- Clonar y arrancar VulnApp en local
- Entender la arquitectura de la app: rutas, middleware, base de datos
- Identificar las vulnerabilidades incluidas intencionalmente
- Realizar el primer ataque básico (exploración manual)

VulnMotors – la app del curso

Stack tecnológico:

Capa	Tecnología
Runtime	Node.js 20+
Lenguaje	TypeScript
Framework	Express
BD	SQLite (better-sqlite3)
Auth	JWT (jsonwebtoken)

Endpoints principales:

Ruta	Función
POST /login	Autenticación
GET /products	Listar coches
PUT /products/:id	Editar coche
GET /users/:id	Perfil usuario
POST /comments	Publicar comentario
GET /debug	Diagnóstico ¡expuesto!

Estructura del código fuente

```
VulnApp/  
├─ src/  
│   ├── routes/  
│   │   ├── auth.ts      ← SQLi (T04), JWT débil (T07)  
│   │   ├── products.ts  ← IDOR (T08), XSS (T05)  
│   │   ├── comments.ts  ← XSS almacenado (T05)  
│   │   ├── users.ts     ← IDOR (T08)  
│   │   └─ debug.ts      ← Command Injection (T06)  
│   ├── middleware/  
│   │   └─ auth.ts       ← alg:none (T07)  
│   └─ db/  
│       └─ database.ts   ← Passwords plain text (T11)  
└─ package.json
```

Usuarios de prueba

Usuario	Contraseña	Rol	Coches propios
alice	password123	admin	Seat Ibiza, Golf GTI, MX-5
bob	qwerty	user	Ford Focus, BMW 320d, Peugeot 208
carol	letmein	user	Renault Clio, Toyota Yaris GR

💀 Contraseñas almacenadas **en texto plano** en la BD — vulnerabilidad T11.

Mapa de vulnerabilidades incluidas

#	Vulnerabilidad	Archivo	OWASP
T04	SQL Injection	<code>auth.ts:30</code>	A03
T05	XSS Almacenado	<code>comments.ts:37</code>	A03
T06	Command Injection	<code>debug.ts:18</code>	A03
T07	JWT débil + alg:none	<code>auth.ts:7</code> , <code>middleware/auth.ts:27</code>	A07
T08	IDOR	<code>products.ts:62</code> , <code>users.ts:10</code>	A01
T11	Passwords en claro	<code>database.ts:57</code>	A02

Cómo arrancar VulnApp

```
# Opción A: directo con Node.js
cd VulnApp
npm install
npm start          # → http://localhost:3000
```

```
# Opción B: con Docker
docker compose up --build
```

💡 La BD se crea automáticamente con datos de prueba la primera vez que arranca.

Primer contacto: exploración manual

```
# Login (obtener token JWT)
curl -X POST http://localhost:3000/login \
  -H "Content-Type: application/json" \
  -d "{\"username\":\"alice\",\"password\":\"password123\"}"

# Listar coches
curl http://localhost:3000/products

# Endpoint de debug (¡sin auth!)
curl http://localhost:3000/debug
```

⚠ Fíjate: `/debug` expone `process.env` sin pedir token — ya es un hallazgo.

Metodología de reconocimiento

📋 1. Enumerar endpoints → 🔍 2. Probar sin auth → 🔑 3. Probar con auth → 📝 4. Documentar hallazgos

Fase	Qué buscar	Herramienta
Enumeración	Rutas, métodos, parámetros	curl, Postman, DevTools
Sin auth	Endpoints abiertos, info leaked	curl sin <code>-H Authorization</code>
Con auth	Diferencias entre roles	Comparar respuestas alice vs bob
Fuzzing	Inputs inesperados	<code>`' " < > ;</code>

Anatomía de un hallazgo de seguridad

```
## Hallazgo: Endpoint de debug sin autenticación

**Severidad:** Alta
**Endpoint:** GET /debug
**Descripción:** Expone todas las variables de entorno (incluyendo
                  JWT_SECRET) sin requerir autenticación.
**Impacto:** Un atacante puede obtener el secreto JWT y firmar
              tokens arbitrarios con cualquier rol.
**Evidencia:** curl http://localhost:3000/debug → {"env":{"JWT_SECRET":"secret",...}}
**Remediación:** Añadir middleware requireAuth + requireAdmin.
```

💡 Documenta cada hallazgo con evidencia reproducible — es la base de un informe profesional.

Herramientas de exploración (REST Client)

En VS Code puedes usar la extensión **REST Client** (`*.http` files):

```
### Login como alice
POST http://localhost:3000/login
Content-Type: application/json

{"username": "alice", "password": "password123"}

### Listar productos (usar token de respuesta anterior)
GET http://localhost:3000/products
Authorization: Bearer {{token}}

### Endpoint de debug
GET http://localhost:3000/debug
```

✓ Más cómodo que curl para probar muchos endpoints. Guarda las peticiones como documentación viva.

Lab T03: Primera exploración de VulnApp

Objetivo: mapear la superficie de ataque de VulnApp

Setup: `cd VulnApp && npm start`

1. Navega por todos los endpoints documentados en `README.md`
2. Intenta acceder a `/debug` sin estar autenticado
3. Explora los parámetros de URL y los cuerpos de formulario
4. Anota qué endpoints tienen validación de input y cuáles no